

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Instytut Elektrotechniki Teoretycznej i Systemów Informacyjno-Pomiarowych

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana
w specjalności Informatyka w Biznesie

Generator tekstowych danych losowych dla zadanego kontekstu wykorzystujący LLM

inż. Jakub Prokopiuk
numer albumu 319002

promotor
dr inż. Radosław Roszczyk

WARSZAWA 2026

Generator tekstowych danych losowych dla zadanego kontekstu wykorzystujący LLM

Streszczenie

Poniższa praca magisterska poświęcona jest zaprojektowaniu i implementacji zaawansowanego systemu do generowania syntetycznych, relacyjnych zbiorów danych, który łączy tradycyjne metody algorytmiczne z możliwościami Dużych Modeli Językowych (LLM).

W ramach pracy opracowano aplikację o architekturze mikroserwisowej, wykorzystującą konteneryzację (Docker) do integracji warstwy frontendowej (React), backendowej (FastAPI) oraz systemu kolejowania zadań asynchronicznych opartego o Redis i Celery. Takie podejście pozwoliło na zapewnienie skalowalności i wydajności niezbędnej do generowania dużych wolumenów danych bez blokowania interfejsu użytkownika.

Kluczowym elementem rozwiązania jest silnik generujący, który implementuje hybrydowe podejście do tworzenia danych: wykorzystuje bibliotekę Faker do danych strukturalnych oraz modele LLM (zarówno chmurowe OpenAI, jak i lokalne, prywatne modele uruchamiane przez Ollama) do generowania treści kreatywnych i kontekstowych. System automatycznie analizuje schemat bazy danych, rozwiązując zależności kluczy obcych (Foreign Keys) w celu zachowania integralności referencyjnej generowanych rekordów.

Zaimplementowane rozwiązanie wykracza poza standardowe generatory plików CSV, oferując mechanizm bezpośredniego zrzutu danych do popularnych silników bazodanowych, takich jak PostgreSQL, MySQL, MSSQL oraz Oracle. Dodatkowo system wyposażono w moduł uwierzytelniania (JWT), zarządzanie projektami w chmurze oraz podgląd postępu generowania w czasie rzeczywistym poprzez protokół WebSocket. Rezultatem pracy jest kompletne, bezpieczne i gotowe do wdrożenia narzędzie, wspierające procesy testowania, prototypowania i analityki danych.

Słowa kluczowe: Generowanie danych syntetycznych, LLM, FastAPI, bazy danych, Docker, Sztuczna Inteligencja, Redis, Celery

Textual Data Generator for a Given Context Using LLM

Abstract

This master's thesis is devoted to the design and implementation of an advanced system for generating synthetic relational datasets, which combines traditional algorithmic methods with the capabilities of Large Language Models (LLMs).

As part of the thesis, an application with a microservice architecture was developed, using containerization (Docker) to integrate the frontend layer (React), backend layer (FastAPI), and an asynchronous task queuing system based on Redis and Celery. This approach ensured the scalability and performance necessary to generate large volumes of data without blocking the user interface.

A key element of the solution is the generation engine, which implements a hybrid approach to data creation: it uses the Faker library for structured data and LLM models (both OpenAI cloud and local, private models run by Ollama) to generate creative and contextual content. The system automatically analyzes the database schema, resolving foreign key dependencies to maintain the referential integrity of the generated records.

The implemented solution goes beyond standard CSV file generators, offering a mechanism for direct data dumps to popular database engines such as PostgreSQL, MySQL, MSSQL, and Oracle. Additionally, the system is equipped with an authentication module (JWT), cloud project management, and real-time generation progress preview via WebSocket protocol. The result of the work is a complete, secure, and ready-to-deploy tool that supports testing, prototyping, and data analytics processes.

Keywords: Synthetic Data Generation, LLM, FastAPI, Databases, Docker, Artificial Intelligence, Redis, Celery

Spis treści

1	Wstęp	9
2	Istniejące rozwiązania	11
2.1	Rozwiązania algorytmiczne i quasi-losowe	11
2.2	Rozwiązania oparte na modelach generatywnych	12
2.3	Rozwiązania oparte na modelach językowych	12
2.4	Proponowane rozwiązanie	12
3	Analiza wymagań i funkcjonalność systemu	15
3.1	Wymagania funkcjonalne	15
3.2	Wymagania нефункционалне	16
	Bibliografia	19
	Wykaz skrótów i symboli	21
	Spis rysunków	23
	Spis tablic	25
	Spis załączników	27

Rozdział 1

Wstęp

We współczesnym świecie dane stanowią jeden z najcenniejszych zasobów przedsiębiorstw, napędzając rozwój systemów informatycznych, analityki biznesowej oraz uczenia maszynowego. Jednakże, wraz ze wzrostem wolumenu przetwarzanych informacji, rośnie również ryzyko związane z ich bezpieczeństwem. W dobie rygorystycznych przepisów o ochronie danych osobowych (GDPR) [1] oraz rosnącej złożoności systemów informatycznych, pozyskanie realistycznych danych testowych, które zachowują spójność logiczną i kontekstową, stanowi istotne wyzwanie inżynierii oprogramowania.

Tradycyjne podejście do testowania oprogramowania często opierało się na wykorzystaniu kopii produkcyjnych baz danych, poddanych procesom anonimizacji lub maskowania. Metody te, choć skuteczne w zachowaniu struktury danych, niosą ze sobą ryzyko reidentyfikacji osób fizycznych [6] oraz są trudne do skalowania w środowiskach Continuous Integration/Continuous Deployment (CI/CD). Z drugiej strony, proste generatory danych losowych (ang. random data generators) często tworzą zbiory niespójne semantycznie – np. generując recenzję produktu, która nie ma związku z jego kategorią, czy adresy niepasujące do kraju użytkownika. Taka niespójność utrudnia testowanie zaawansowanych algorytmów analitycznych i systemów rekomendacyjnych.

Przełomem w dziedzinie generowania treści stało się upowszechnienie Dużych Modeli Językowych (LLM, ang. Large Language Models). Modele takie jak GPT-4 czy Llama 3 potrafią generować tekst o wysokim stopniu realizmu, uwzględniając zadany kontekst [5]. Jednak samo użycie LLM jest procesem kosztownym obliczeniowo i wolnym, co czyni go niepraktycznym przy generowaniu milionów rekordów bazy danych. Istnieje zatem zapotrzebowanie na rozwiązania hybrydowe, łączące szybkość klasycznych algorytmów z kreatywnością sztucznej inteligencji.

Celem niniejszej pracy jest zaprojektowanie i implementacja narzędzia służącego do proceduralnego generowania relacyjnych zbiorów danych. Proponowane rozwiązanie integruje wydajne algorytmy pseudolosowe (biblioteka Faker) do generowania danych strukturalnych oraz modele LLM do tworzenia danych kontekstowych. System został zaprojektowany w architekturze mikroserwisowej, wykorzystując konteneryzację Docker, kolejkę zadań Redis oraz asynchroniczne przetwarzanie, co pozwala na symulację środowisk produkcyjnych o wysokiej złożoności.

Rozdział 2

Istniejące rozwiązania

Rynek narzędzi do generowania danych syntetycznych jest obecnie dynamicznie rozwijającym się obszarem, który można podzielić na trzy główne kategorie: rozwiązania algorytmiczne, rozwiązania oparte na uczeniu maszynowym oraz nowo powstające rozwiązania oparte o modele językowe (LLM). Poniższa analiza przedstawia wiodące narzędzia w tych kategoriach, wskazując ich zalety oraz ograniczenia w kontekście generowania spójnych relacyjnie baz danych.

2.1 Rozwiązania algorytmiczne i quasi-losowe

Jeszcze do niedawna były to najpopularniejsze narzędzia do generowania danych syntetycznych. Sztandarowe narzędzia w tej kategorii to m.in.:

Biblioteka Faker

Faker [2] to biblioteka open-source dostępna w wielu językach programowania (Python, JavaScript, Ruby), umożliwiająca generowanie różnorodnych danych losowych, takich jak imiona, adresy, numery telefonów czy dane finansowe.

- **Zalety:** Bardzo wysoka wydajność (generowanie odbywa się w czasie rzeczywistym), brak kosztów (narzędzie jest open-source), determinizm (możliwość odtworzenia danych przy użyciu seed).
- **Wady:** Brak kontekstualnej spójności między generowanymi danymi. Wygenerowana "recenzja produktu," to zazwyczaj zlepek losowych słów z łaciny lub zdań niemających sensu logicznego. Znaczącą wadą jest również brak wbudowanej obsługi złożonych relacji między tabelami (użytkownik musi sam zaprogramować logikę kluczy obcych).

Mockaroo

Mockaroo to popularne narzędzie webowe (SaaS), które pozwala definiować schematy danych w GUI i pobierać je jako CSV/SQL/JSON.

- **Zalety:** Intuicyjny interfejs użytkownika, szeroki zakres typów danych, możliwość eksportu do różnych formatów, wsparcie dla relacji między tabelami za pomocą prostych skryptów pisanych w Ruby.
- **Wady:** W wersji darmowej generowanie limitowane jest do 1000 wierszy. Mimo zaawansowania, nadal opiera się głównie na losowaniu ze słowników. Funkcje sztucznej inteligencji są ograniczone lub dodatkowo płatne.

2.2 Rozwiązania oparte na modelach generatywnych

Ta grupa narzędzi (np. Gretel.ai [3], Mostly AI [4]) posiada duże problemy z prywatnością danych. Działają one na zasadzie: "Wgraj nam swoją prawdziwą bazę danych, a my nauczymy model jej schematu i wygenerujemy nową, która wygląda tak samo, ale nie zawiera danych osobowych,,.

- **Zalety:** Potrafią generować dane o wysokim stopniu realizmu, zachowując statystyczne właściwości oryginalnego zbioru.
- **Wady:** Wymagają dostępu do rzeczywistych danych, co może naruszać przepisy o ochronie danych osobowych. Wymagają również danych wejściowych przez co nie nadają się do tworzenia danych do nowo powstających systemów, gdzie nie ma jeszcze żadnej bazy danych, na której można by uczyć model. Koszty licencyjne są wysokie, a skalowalność ograniczona przez infrastrukturę dostawcy.

2.3 Rozwiązania oparte na modelach językowych

Najnowsza kategoria narzędzi do generowania danych syntetycznych wykorzystuje możliwości Dużych Modeli Językowych, takich jak GPT-4 czy Claude. Przykładowo, możemy poprosić model o wygenerowanie danych do tabeli "Recenzje Produktów,, , podając mu kontekst (np. kategorie produktów, cechy użytkowników).

- **Zalety:** Wysoki realizm tekstowy, kreatywność, rozumienie kontekstu.
- **Wady:** Wysokie koszty operacyjne, ograniczenia szybkości, model może wymyślić nieistniejące ID lub zgubić strukturę JSON/SQL przy dużej ilości danych. Przez ograniczenia kontekstowe, generowanie dużych zbiorów danych wymaga podziału na mniejsze partie, co komplikuje zachowanie spójności referencyjnej między tabelami.

2.4 Proponowane rozwiązanie

Analiza powyższych rozwiązań wskazuje na istotną lukę rynkową. Brakuje narzędzia, które:

- nie wymaga danych wejściowych,
- zapewnia wysoki realizm i kontekstowość danych tekstowych,
- gwarantuje ścisłą integralność relacyjną przy dużej skali,

- jest efektywne kosztowo (nie generuje prostych danych typu "data urodzenia,, przez drogie LLM).

Rozdział 3

Analiza wymagań i funkcjonalność systemu

Proces wytwarzania oprogramowania wymaga precyzyjnego zdefiniowania założeń projektowych. W przypadku systemów wykorzystujących generatywną sztuczną inteligencję, kluczowe jest wyznaczenie granic między deterministycznym działaniem algorytmów a probabilistyczną naturą modeli językowych. Poniższy rozdział definiuje specyfikację funkcjonalną i нефункциональную projektowanego systemu, stanowiącą fundament dla implementacji architektury mikroservisowej opisanej w kolejnych częściach pracy.

3.1 Wymagania funkcjonalne

Głównym celem systemu jest umożliwienie użytkownikom generowania syntetycznych, relacyjnych zbiorów danych o wysokim stopniu realizmu, z wykorzystaniem hybrydowego silnika łączącego metody regułowe i modele LLM.

Definiowanie schematów i typów danych

System powinien udostępniać graficzny interfejs do modelowania struktury bazy danych. Użytkownik powinien mieć możliwość definiowania tabel, kolumn oraz typów danych, wybierając spośród trzech głównych generatorów:

- **Generator deterministyczny:** Powinien służyć do szybkiego tworzenia standardowych danych o przewidywalnej strukturze (np. imiona, adresy, daty, identyfikatory UUID).
- **Generator kontekstowy:** Powinien pozwalać na zdefiniowanie szablonu promptu (np. „Napisz e-mail reklamacyjny dotyczący zamówienia {order_id}”), który następnie zostanie przetworzony przez model językowy w celu wygenerowania unikalnej treści.
- **Generator dystrybucyjny:** Powinien umożliwiać zdefiniowanie ważonych list wartości (np. statusy zamówień z określonym prawdopodobieństwem wystąpienia), aby odzwierciedlić rzeczywisty rozkład danych.

Zachowanie integralności relacyjnej [8]

Krytyczną funkcjonalnością systemu jest automatyczne zarządzanie relacjami między tabelami. Aplikacja powinna pozwalać na definiowanie kluczy obcych [7], a silnik generujący ma za zadanie automatycznie budować graf zależności i ustalać topologiczną kolejność generowania danych. Mechanizm ten musi gwarantować, że rekordy w tabelach podrzędnych będą zawsze odwoływać się do istniejących rekordów w tabelach nadrzędnych, eliminując błędy spójności.

Orkiestracja zadań generowania

Ze względu na przewidywaną czasochłonność operacji z użyciem LLM, system powinien umożliwiać asynchroniczne uruchamianie procesów generowania. Użytkownik powinien mieć możliwość zlecenia zadania, które trafi do kolejki przetwarzania, a system musi zapewniać mechanizm podglądu postępu prac w czasie rzeczywistym.

Bezpośrednia integracja z bazami danych

Poza standardowym eksportem do plików płaskich (JSON, CSV) czy eksportu do poleceń INSERT, system powinien oferować funkcję bezpośredniego przesyłania do zewnętrznych baz danych. Aplikacja powinna obsługiwać połączenia z najpopularniejszymi silnikami SQL (PostgreSQL, MySQL, MSSQL, Oracle) i wykonywać operacje typu *batch insert*. Pozwoli to na natychmiastowe wykorzystanie wygenerowanych danych w środowiskach testowych lub deweloperskich bez konieczności ręcznego importu plików.

Zarządzanie projektami i szablonami

Zalogowani użytkownicy powinni mieć możliwość zapisywania zdefiniowanych schematów w chmurze, co pozwoli na ich późniejszą edycję i ponowne wykorzystanie. System powinien również wspierać import i eksport konfiguracji, ułatwiając współdzielenie schematów w zespołach deweloperskich.

3.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają atrybuty jakościowe systemu, które są niezbędne do jego wdrożenia w środowisku produkcyjnym, ze szczególnym naciskiem na wydajność, bezpieczeństwo i skalowalność.

Architektura asynchroniczna i wydajność

System musi być zdolny do obsługi długotrwałych procesów generowania danych bez blokowania interfejsu użytkownika. W tym celu należy zastosować architekturę opartą na kolejce komunikatów, która odseparuje warstwę API od warstwy wykonawczej. Rozwiązanie to powinno zapewniać stabilność

działania nawet przy generowaniu zbiorów liczących dziesiątki tysięcy rekordów, a także odporność na chwilowe błędy dostępności API zewnętrznych.

Prywatność danych i obsługa modeli lokalnych

W odpowiedzi na wymogi bezpieczeństwa, system powinien zapewniać możliwość działania w trybie całkowicie odciętym od sieci zewnętrznej [9]. Wymagane jest umożliwienie integracji z lokalnymi modelami LLM, co zagwarantuje, że żadne dane ani prompty nie opuszczą infrastruktury wewnętrznej użytkownika.

Skalowalność horyzontalna

System powinien być zaprojektowany z myślą o skalowaniu horyzontalnym, wykorzystując konteneryzację. Pozwoli to na łatwe dostosowanie zasobów obliczeniowych do rosnącego zapotrzebowania, poprzez uruchamianie dodatkowych instancji mikroservisów w środowiskach chmurowych lub klastrach kontenerowych.

Bezpieczeństwo dostępu

Dostęp do zasobów systemu musi być chroniony mechanizmem uwierzytelniania. Hasła użytkowników powinny być przechowywane w bazie danych wyłącznie w formie zaszyfrowanej, a cała komunikacja między komponentami systemu oraz z zewnętrznymi bazami danych powinna odbywać się przy użyciu bezpiecznych protokołów.

Rozszerzalność i modularność

Architektura systemu powinna zostać zaprojektowana w sposób modułowy. Ma to na celu umożliwienie łatwego dodawania nowych konektorów baz danych oraz integrację z nowymi dostawcami modeli AI w przyszłości, bez konieczności gruntownej przebudowy rdzenia aplikacji.

Bibliografia

- [1] European Parliament and Council of the European Union, *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*, 2016. adr.: <https://data.europa.eu/eli/reg/2016/679/oj>.
- [2] Faker-js, *Faker*, wer. 9.3.0, Dostęp: 2026-01-10, 2024. adr.: <https://github.com/faker-js/faker>.
- [3] Gretel Labs, Inc. „Gretel: The Synthetic Data Platform for Privacy-Preserving AI.” Dostęp: 2026-01-10. (2024), adr.: <https://gretel.ai>.
- [4] MOSTLY AI. „MOSTLY AI: The Synthetic Data Platform.” Dostęp: 2026-01-10. (2024), adr.: <https://mostly.ai>.
- [5] Nadăș, M., Dioșan, L. i Tomescu, A., „Synthetic Data Generation Using Large Language Models: Advances in Text and Code,” *IEEE Access*, t. 13, s. 134 615–134 633, 2025, ISSN: 2169-3536. DOI: 10.1109/access.2025.3589503. adr.: <http://dx.doi.org/10.1109/ACCESS.2025.3589503>.
- [6] Ravens, E. L. T. .-, *Data Privacy — The Netflix Prize competition*, <https://medium.com/@EmiLabsTech/data-privacy-the-netflix-prize-competition-84330d01cc34>, Accessed: 2026-01-03, 2022.
- [7] Wikipedia contributors, *Foreign key — Wikipedia, The Free Encyclopedia*, https://en.wikipedia.org/w/index.php?title=Foreign_key&oldid=1270418735, [Online; accessed 10-January-2026], 2025.
- [8] Wikipedia contributors, *Referential integrity — Wikipedia, The Free Encyclopedia*, https://en.wikipedia.org/w/index.php?title=Referential_integrity&oldid=1308130831, [Online; accessed 10-January-2026], 2025.
- [9] Wikipedia contributors, *Air gap (networking) — Wikipedia, The Free Encyclopedia*, [https://en.wikipedia.org/w/index.php?title=Air_gap_\(networking\)&oldid=1331234099](https://en.wikipedia.org/w/index.php?title=Air_gap_(networking)&oldid=1331234099), [Online; accessed 10-January-2026], 2026.

Wykaz skrótów i symboli

Spis rysunków

Spis tablic

Spis załączników